

AROL CUSTOMER PLATFORM

Daniele Bongiovanni s358622 - Alice Buccoleri s358621

1. Aim of the Project

The **AROL Customer Platform** is a web application for authorised users of AROL customer companies, including operators, technicians, and commercial contacts.

It centralises machine-fleet information in a single authenticated interface, while enforcing company-level and role-based access permissions.

Users can view the machines assigned to their company, including configuration and delivery details, IoT telemetry snapshots, alarm history, maintenance tickets, quotations, orders, and machine-specific technical manuals. A QR-code lookup can open the authorised context of a specific machine directly.

The platform supports both direct **consultation** through tables and detail pages and natural-language questions through the AROL Assistant. Depending on the user's permissions, the assistant can retrieve information about machine operation and alarms, maintenance history, manuals, quotations, and orders.

Results may include structured records for the interface and cited excerpts from the relevant manuals.

Access to platform **data** is governed by the user's organisation and visibility permissions.

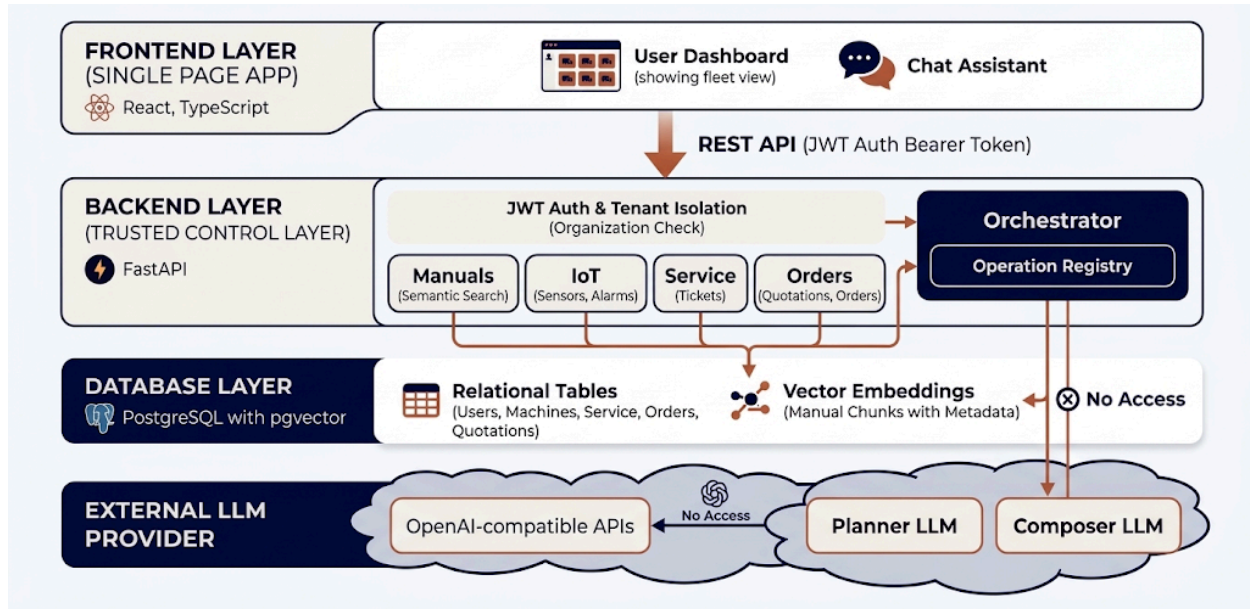
The conversational assistant uses a language-model-based orchestrator to coordinate specialised Manuals, IoT, Service and Orders agents.

The orchestrator selects authorised operations, gathers fresh evidence from the relevant sources, correlates that evidence, and produces prudent responses grounded in available records and cited manual excerpts.

It does not treat conversational context as evidence and does not invent procedures, causes or maintenance conclusions.

2. System Architecture

The architecture of the AROL Customer Platform consists of **four main components**: the frontend, the backend, the database and an external language-model provider.



1. The **frontend** is developed using React and TypeScript. It provides the graphical interface through which authenticated users can browse machine information, consult tables and documents, select a machine context, and interact with the conversational assistant.
2. The **backend** is built with FastAPI and acts as the trusted control layer of the platform. It handles authentication, organisation and visibility permissions, machine context, API requests and access to internal resources.
3. **PostgreSQL** stores the platform's structured data, including users, machines, telemetry, alarm records, service tickets, orders and quotations. It is integrated with pgvector to store and search vector embeddings associated with authorised manual chunks. A local embedding model is used to support retrieval-augmented generation over the documentation.
4. The **external language-model provider** is used only through controlled backend requests. The model has no autonomous access to databases, documents or internal

application functions. When language-model processing is required, the backend selects and provides the authorised data needed for the specific request.

2.1 Frontend architecture

The **frontend** is implemented as a single-page application using React, TypeScript and Vite. It provides the user-facing layer of the platform and communicates with the FastAPI backend through REST APIs. After successful authentication, the application stores the user session locally and includes the bearer token in requests to protected backend endpoints.

The interface is organised into **dedicated pages** for the main platform functions:

- The home page displays the fleet of machines available to the authenticated user.
- A machine page presents the selected machine context, including identification details, configuration, delivery information, operational context, telemetry, alarms, maintenance tickets and manual search.
- The application also provides separate pages for orders, quotations, quotation revision history, after-sales service tickets and the user profile. These are accessible via the app header.

The frontend **source code** is organised into dedicated folders to separate responsibilities and simplify maintenance. Pages contain the main views, while reusable components, API logic, authentication handling, types, utilities, and styles are kept in separate modules.

A machine can be selected from the fleet view or resolved through its QR code.

The QR scanner reads the code and directs the user to the corresponding machine page: the backend resolves the identifier and verifies that the machine belongs to the authenticated user's organisation before returning its context.

The main component of the platform is the **Assistant**, available from the machine page and already connected to the current machine identifier when the user opens the chat.

It displays a response generated by the backend together with structured results.

When a response is based on technical documentation, the interface renders the available

manual sources as citation cards containing the document title, relevant excerpt, file and page reference.

The **visual design** of the application was inspired by the style of the AROL corporate website. The interface uses a clean industrial aesthetic, with dark-blue identity and card-based information panels.

To conclude, the frontend manages **user feedback** for loading, empty, access-denied and error states.

2.2 Back-end architecture

The **backend** is implemented with FastAPI and acts as the trusted control layer of the AROL Customer Platform. It exposes the REST API used by the frontend, manages authenticated requests, resolves the user's organisation and visibility profile, validates machine context, and coordinates the retrieval of information from the platform's internal data sources.

All security-sensitive decisions, including tenant isolation, role-based visibility, machine ownership, and operation validation, are enforced by backend components rather than by the frontend or the language model.

Conversational requests are managed by an **orchestrator**.

The orchestrator is the only component that interacts with the external language-model provider. It uses the LLM in two separate and constrained roles:

- The **Planner LLM** interprets the user's request and produces a structured decision containing either a direct answer request or a plan of evidence-retrieval operations.
- After the requested evidence has been collected and validated, the **Composer LLM** generates the final natural-language response. The LLM does not have direct access to the database, PDF manuals, vector index, internal APIs, or backend functions.

The plan produced by the Planner LLM is not executed directly.

It is parsed and validated against backend-owned **Pydantic contracts**, which enforce the expected structure, allowed fields, and typed parameters of each request.

The backend also verifies any machine identifier and ensures that it is compatible with the

authenticated user's organisation and visibility profile. A malformed plan, an unknown operation, an unauthorised parameter, or missing trusted machine context is rejected before any data-access operation is executed.

The available operations are defined by the **Operation Registry**.

The registry acts as an internal gateway between the LLM-generated plan and the application services. For each registered operation, it defines the responsible domain module, the Pydantic parameter model, whether trusted machine context is required, and the authorised handler that performs the retrieval. The registry therefore ensures that the Planner LLM can select only explicit and pre-defined operations; it cannot generate arbitrary SQL queries, invoke internal functions freely, select an organisation, or bypass visibility checks.

This project adopts a **custom, backend-owned orchestration layer** rather than a general-purpose agent framework such as LangGraph or AutoGen.

In this way, the conversational workflow is defined and guided by the requests: each request follows a controlled sequence of planning, validation, evidence retrieval, evidence formatting, and response composition.

Implementing the orchestration directly in the backend keeps these controls close to the API and data-access layers, where they can be validated and tested deterministically.

General-purpose frameworks could provide useful capabilities such as persistent workflow state, retries or more complex agent handoffs.

However, we believe it would introduce additional abstraction without replacing the security and authorisation logic.

The Manuals, IoT, Service, and Orders modules are specialised **evidence-retrieval modules**, rather than autonomous AI agents. They do not independently interpret the user's request, decide which operations to perform, invoke other modules, or call the language model.

Each module receives an already validated request from the Operation Registry and retrieves evidence only within its specific domain. The Manuals module performs local semantic retrieval over machine-specific manual excerpts; the IoT module retrieves telemetry and alarm records; the Service module retrieves maintenance-ticket information; and the Orders module retrieves quotations, orders, and fulfilment data.

Each module accesses data through controlled **data-access functions**.

These functions use parameterised SQL queries and apply organisation, visibility, and machine filters using the trusted authentication context created by the backend.

The retrieved data is then processed by the **Evidence Formatters**, which convert the raw domain results into standardised AgentResult objects, which can contain authorised factual evidence, warnings, citations, and structured data intended for frontend tables. In the case of technical manuals, the formatter removes local-only raw chunks and provides only a bounded set of relevant excerpts with document and page references.

The AgentResult objects are collected in an **EvidenceBundle**.

This structure separates the evidence used for language-model composition from the structured data returned to the frontend. The orchestrator can correlate evidence from multiple domains when a request requires it, for example, by combining alarm records, maintenance tickets, and manual excerpts. It then prepares a limited set of authorised evidence for the Composer LLM. The Composer LLM produces a coherent response grounded exclusively in this evidence, while the frontend receives the final text together with structured records and citations where applicable.

This architecture ensures that conversational responses remain evidence-based and permission-aware. The Planner LLM can propose only registered operations, the backend validates and executes those operations, and the Composer LLM receives only authorised evidence after retrieval. The custom orchestration layer therefore provides a controlled and auditable implementation of the agentic workflow required by the platform.

2.2.1 Design Choice

The implementation does not use an external **Model Context Protocol (MCP) server** or expose tools through the MCP network protocol.

Instead, the Operation Registry provides an internal mechanism that is conceptually similar to the control layer offered by MCP: it maintains an operation catalogue, validates schemas and parameters, applies policy checks, and dispatches authorised requests to controlled handlers. This is not a formal MCP implementation, but it preserves the relevant architectural properties for the prototype: explicit tool-like operations, typed interfaces, centralised authorisation,

controlled execution, and a strict boundary between the LLM and internal business data. This approach is appropriate for a closed system composed of one frontend, one backend, internal modules, and sensitive company data. A standard MCP integration could be considered in the future if the same operations needed to be exposed to multiple external clients, agents, or language-model providers.

2.2.2 Local Retrieval-Augmented Generation (RAG) for Manuals

The Manuals module implements a local **Retrieval-Augmented Generation (RAG)** workflow for technical documentation. PDF manuals are processed locally and their textual content is extracted page by page.

It is divided into section-aware chunks with overlap in order to preserve the context between consecutive passages. Each chunk is associated with metadata identifying the relevant machine, source file, page number, document section and a stable chunk identifier.

For each chunk, the system generates a vector embedding using the local sentence-transformers/all-MiniLM-L6-v2 model. The embeddings and the associated metadata are then stored in the manual_chunks table in PostgreSQL, using the pgvector extension. Semantic retrieval is performed through an HNSW index and cosine-distance similarity search. This makes it possible to identify documentation passages that are semantically related to a user's question, even when the wording used in the request differs from the wording contained in the manual.

When the Planner LLM selects a manual-retrieval operation, the backend first verifies the authenticated user's organisation, visibility profile, and the machine context.

The search is then restricted to chunks belonging to manuals authorised for that specific machine. The initial semantic-search results are re-ranked locally according to their relevance to the request.

Raw retrieved manual chunks are not sent directly to the frontend or to the external LLM. Instead, the Evidence Formatter creates a bounded set of authorised excerpts, each accompanied by its document title, file name, page number, relevance information, and citation.

Only these selected excerpts are made available to the Composer LLM as evidence for the final answer.

2.2.3 Semantic Capability Retrieval for the Planner

Before invoking the Planner LLM, the backend performs **local semantic retrieval** over the catalogue of authorised operations.

Each operation registered in the Operation Registry is represented by a capability description containing its agent, operation name, purpose, parameter schema, and machine-context requirement. These descriptions are embedded locally using the same sentence-transformers/all-MiniLM-L6-v2 model used for manual retrieval and are stored in the operation_capabilities table in PostgreSQL with the pgvector extension.

When the user submits a question, the backend generates a **local embedding** of the request and performs a cosine-similarity search over the capability catalogue.

The retrieved capability entries are used to build a smaller, semantically relevant operation catalogue for the Planner LLM. This reduces the number of operations exposed in the planning prompt and helps the model select the appropriate domain and operation for the request.

Capability retrieval does not provide the Planner LLM with factual evidence, tenant data, manual content, or database records. It only ranks descriptions of operations that are already authorised by the backend.

Each retrieved entry is reconstructed from the live Operation Registry before being added to the planner prompt; therefore, an outdated or modified database entry cannot introduce a new operation or bypass the registry's validation rules. If semantic capability retrieval is unavailable or returns no suitable candidates, the backend falls back to the complete registry catalogue. The manuals.search operation is also retained when necessary to ensure that documented evidence remains available for technical questions.

2.3 Databases and data management

The platform uses PostgreSQL, integrated with the pgvector extension, to manage both the application's structured data and the vector representations used for document retrieval.

Each **machine** is associated with a specific organisation. This relationship forms the basis of application-level tenant isolation and allows the backend to apply the filters required to ensure that users can access only resources belonging to their own organisation.

These checks are performed during data access using the authenticated context created by the backend for each request.

Database access is managed through a dedicated **data-access layer**, which defines the SQL queries, organisation and machine filters, counts, minimal record transformations and list-completeness metadata.

The queries are parameterised and executed exclusively by controlled backend components.

The language model therefore cannot autonomously generate or execute arbitrary SQL queries and may only request the authorised operations exposed by the application.

2.4 Conversational Assistant and Memory

To improve continuity in interactions with the assistant, a temporary and controlled **conversational memory** has been introduced, allowing the chat to interpret questions that depend on previous messages. The conversation history is used exclusively to complete the meaning of a new request and is never treated as a factual source for the response.

Any required information is always retrieved again by the agents from authorised sources.

When the user submits a new question, the frontend sends the backend the new message, the identifier of the selected machine, if applicable, and the last eight completed messages in the conversation. For each message, only its role, either user or assistant, and its textual content are transmitted.

The **conversation history** does not include structured data, manual sources, raw results, citations or evidence retrieved during previous requests. It is also not stored in the database and is deleted together with the conversation state when the user closes the chat.

The backend receives the history through the ConversationTurn contract, which validates the number of messages, their roles, the correct role alternation and the length of their content.

The first implementation involved two separate stages.

- The conversation history was initially used to rewrite the user's question, after which a second planner analysed the new formulation and selected the operations to be executed. However, important references could be lost between the two stages.
- To solve this problem, the two-stage process was replaced by a **single context-aware planner**. When conversational memory is required, the planner receives the recent conversation history, the new question, the reference date and the catalogue of operations authorised by the Operation Registry.

It then produces a structured object specifying the action to be taken, the identified intent, the references extracted from the conversation and the plan to be assigned to the agents. A reference inherited from the conversation history must not only be recognised but also included in the parameters of the selected operations.

The plan produced by the model is subject to **server-side checks**.

The backend verifies that the identifiers specified by the planner are actually present in the question or conversation history, that they are included in the plan parameters and that all requested operations belong to the Operation Registry.

It also validates the parameter structure and the set of operations required for each type of request. As a result, the model cannot invent an identifier or declare a specific reference and then perform a generic search.

The conversation history is used only when the question contains references that depend on **previous context**. Requests such as *"Can you tell me more about that alarm?"* or *"How many times did it occur on 2026-07-30?"* require conversational memory to identify the referenced object correctly.

By contrast, a question that already contains an explicit code, such as AL083_FALLEN_BOTTLE_ALARM, or introduces a new topic, such as *"What is the current production rate?"*, is processed without providing the history to the planner. This distinction reduces the risk of previous but no longer relevant references affecting the new plan. The mechanism determines only whether conversational context is required, while intent recognition and operation selection remain the responsibility of the planner, the Operation Registry and the specialised agents.

2.5 Authentication and Access Control

Access to the platform is protected through **JWT-based authentication**.

During login, the backend verifies the user's credentials and issues a short-lived bearer token.

For each subsequent request, the backend validates the token signature and expiry, extracts the user identifier, and retrieves the current organisation and visibility level from the database.

It then creates a trusted authentication context containing the user identifier, organisation identifier and visibility level.

A **machine context** may be selected from the machine page, resolved from a QR code, or resolved from an identifier included in a conversational request.

The backend treats any user-provided identifier only as a candidate and verifies access before machine-specific data can be retrieved.

Authorisation rules are enforced at **multiple levels**, including API endpoints, the operation registry, specialised agents and the data-access layer.

User role	Accessible information
Full visibility user	Machine information, operational data, technical manuals, maintenance tickets, quotations, and orders.
Technician user	Machine information, IoT telemetry, alarms, maintenance tickets, and technical manuals. Commercial records are not available.
Commercial user	Machine information, technical manuals, quotations, and orders. Operational data and maintenance tickets are not available.

All **users** can access only the machines associated with their own organisation.

The visible interface and the operations available through the AROL Assistant are determined

by the authenticated user's visibility profile.

A request for information outside the permitted scope is explicitly denied rather than returned as an empty result.

The **language model** cannot modify the authenticated user context, choose a trusted organisation or machine, or grant additional privileges. Any plan proposed by the model is validated by the backend and operation registry before an agent operation is executed.

3. Deployment and Technology Stack

The AROL Customer Platform is deployed locally through **Docker Compose**.

This approach provides a reproducible development environment and separates the main runtime services while keeping the deployment topology simple.

The application is composed of a frontend container, a backend container, and a PostgreSQL database container. An additional Adminer container is available exclusively as a development tool for inspecting the database through a browser and is not part of the logical application architecture.

1. The frontend is implemented with **React 18**, **TypeScript**, and **Vite**.

It runs in a node:20-alpine container, while React Router is used to manage the views available to the user, including login, fleet, machine, order, quotation, service, and profile pages.

2. The backend is developed in **Python 3.12** using **FastAPI** and runs in a container based on the python:3.12-slim image. The API service is started with Uvicorn and depends on the PostgreSQL service health check before becoming available.
3. The database layer uses **PostgreSQL 16** with the **pgvector** extension.

Database data is persisted through a Docker volume, while schema creation and extension initialisation are performed through SQL scripts mounted during database initialisation.

Technical manuals remain in the local data/manuals directory and are mounted read-only inside the backend container. The downloaded embedding model is persisted in a dedicated Docker volume, allowing it to be reused across container restarts without being downloaded again.

The manual PDFs, extracted raw chunks, and vector index are therefore kept inside the local environment.

The language model is accessed through an OpenAI-compatible API configured through environment variables, including the provider endpoint, API key, and model name.

In the current project, the integration is configured for Groq.

The backend is the only component allowed to contact the external provider: it sends structured planning requests and bounded, authorised evidence for response composition.

Runtime configuration, including database credentials, JWT secrets, development passwords, and LLM settings, is provided through a local `.env` file.

This file, together with the source dataset and manuals, is excluded from version control.

The complete platform can be started through a single Docker Compose command, while dedicated commands are available for importing the dataset, creating development passwords, building the manual index, and evaluating local RAG performance.

4. Testing and Validation

The platform was validated through automated backend, integration, frontend, RAG and conversational evaluations.

- The **backend test suite** verifies authentication, JWT handling, organisation isolation, role-based visibility, API contracts, operation validation, orchestration behaviour, alarms, maintenance tickets, quotations, orders, and manual retrieval.
- **Integration tests** run against a disposable PostgreSQL database using the same schema migrations as the development environment.
- **Frontend validation** includes TypeScript compilation, Vite production build and Playwright end-to-end tests.

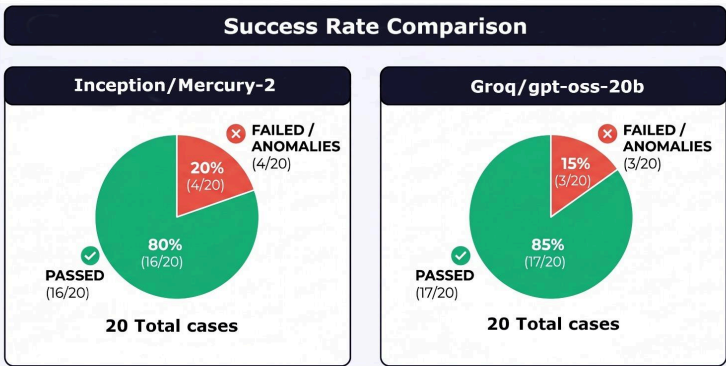
All automated software test suites passed successfully.

The **local RAG evaluation** was conducted on 16 manual-retrieval cases.

It achieved a File Recall@K of 1.00 and a Mean Reciprocal Rank of 1.00, showing that the relevant manual file was consistently retrieved and ranked correctly.

Mean keyword coverage was 71.4%. Page-level retrieval obtained a Page Recall@K of 75%, increasing to 93.8% when the 2 adjacent pages were considered acceptable. These results indicate that the retrieval process is effective at document level and demonstrates strong accuracy at the page level, successfully retrieving the exact or adjacent pages in the vast majority of cases.

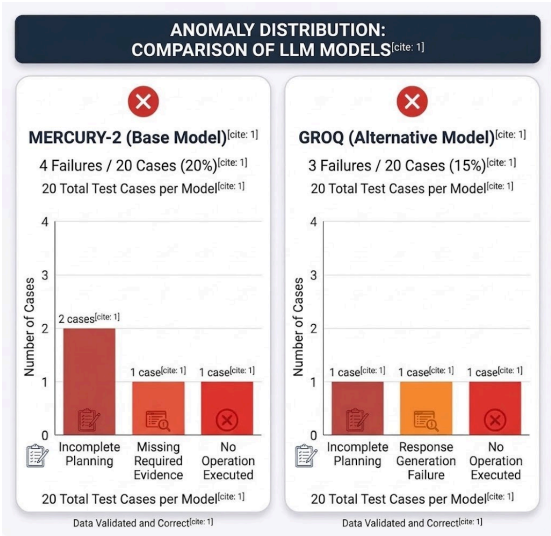
A conversational benchmark was then executed on 20 test cases for each **evaluated LLM provider**. The benchmark assessed the ability of the system to produce a correct evidence-retrieval plan, execute the required operations, and generate a grounded final response.



Response Latency Comparison (Response Latency Benchmark)

Metric	Mercury-2 (Base Model)	Groq (openai/gpt-oss-20b)
Average Latency	3.41 s[cite: 1]	27.69 s[cite: 1]
Fastest Time (Min)	1.48 s[cite: 1]	7.99 s[cite: 1]
Slowest Time (Max)	10.64 s[cite: 1]	48.30 s[cite: 1]

Note for report: Mercury-2 is approximately 8 times faster on average, fundamental for an interactive chatbot experience.



The results show that the main remaining limitation concerns the **reliability of the LLM-driven orchestration workflow**. Incomplete planning occurs when the model does not request every operation needed to answer a question completely.

Missing required evidence occurs when the answer lacks an essential authorised source, while a no-operation-executed case indicates that the workflow didn't reach a valid retrieval operation.

These failures are limited in number but remain relevant because the platform handles technical and operational information.

5. Conclusion

The AROL Customer Platform demonstrates how fleet information, operational data, commercial records, technical manuals, and conversational interaction can be **integrated** into a single authenticated web application.

A central **design objective** was to maintain a strict security boundary between the language model and internal company data. The LLM cannot access databases, manuals, SQL queries, or internal functions directly. These guarantees are supported by a custom backend-owned orchestration layer.

The **evaluation** confirms that the local RAG pipeline retrieves the relevant manual files reliably. Furthermore, a conversational benchmark conducted on a dataset of 20 specific test cases shows that both evaluated models can handle most requests successfully.

The openai/gpt-oss-20b model achieved the highest measured success rate at 85%, while Mercury-2 achieved 80% with a substantially lower mean latency of 3.41 seconds.

However, the prototype has not yet reached fully reliable behaviour for every complex conversational request. The remaining errors concern incomplete planning, missing evidence, response generation, and cases in which no retrieval operation is executed.

Future work will focus on evaluating stronger reasoning models, improving planner prompts and multi-source evidence representation, adding review steps for complex requests, and repeating the benchmark after each orchestration improvement. The backend will remain the deterministic security and authorisation boundary regardless of the selected LLM provider.